

C2 Evasion RL: Packet-Level Reinforcement Learning for Adversarial Network Flow Evasion

Lê Hoàng Phát

September 25, 2026

Abstract

This thesis investigates how reinforcement learning (PPO) can be used to craft adversarial network flows that evade detection by machine learning-based intrusion detection systems (IDS). We train an agent on a surrogate XGBoost model to mutate botnet C2 flows from the CTU-13 dataset, and validate evasion success against a real Snort 2.9 IDS running the Emerging Threats Open ruleset. Key findings: (1) packet-level mutations achieve 100% evasion on small flow sets (24 flows, cost=0.6) against Snort; (2) evasion degrades non-monotonically with batch size, peaking at 95.7% on 300 flows; (3) the surrogate-to-real gap persists (95% XGBoost evasion vs 32.5% Snort evasion for the baseline agent), indicating that surrogate-based training does not transfer to real detection logic. We document critical bugs in flow extraction and packet rewriting that initially prevented real Snort detection, and provide reproducible measurements across three experimental phases: corrupt-cost sweep, flow scale-up, and cross-capture validation.

Contents

1	Introduction	3
1.1	Motivation	3
1.2	Problem Statement	3
1.3	Contributions	4
2	Threat Model & Scope	5
2.1	Adversarial Setting	5
2.1.1	Red Team (Attacker)	5
2.1.2	Blue Team (Defender)	5
2.2	Out of Scope	6
3	System Architecture	7
3.1	Pipeline Overview	7
3.1.1	Data Flow	7
3.2	Environment Design	7
3.2.1	Observation Space	7
3.2.2	Action Space	8
3.2.3	Reward Function	8
3.3	Snort Integration	8
4	Mathematical Formulation	9
4.1	Markov Decision Process	9
4.2	Policy Gradient Optimization	9
4.3	Surrogate Training	9
5	Experiments	10
5.1	Dataset: CTU-13 & Stratosphere IPS	10
5.2	Experimental Phases	10
5.2.1	Phase 1: Surrogate Training & Baseline	10
5.2.2	Phase 2: PPO Agent Training	10
5.2.3	Phase 3: Stratosphere Validation	11
5.3	Results Summary	11
5.3.1	Primary Finding: 100% Deterministic Evasion	11
5.3.2	Secondary Finding: Non-Monotonic Scaling	11
5.3.3	Tertiary Finding: Massive Surrogate Gap	11

6	Challenges & Lessons	12
6.1	Bug #1: Unidirectional Flow Extraction	12
6.1.1	Symptom	12
6.1.2	Root Cause	12
6.1.3	Fix	12
6.1.4	Verification	12
6.2	Bug #2: Naive Packet Rewriting	12
6.2.1	Symptom	12
6.2.2	Root Cause	13
6.2.3	Fix	13
6.2.4	Verification	13
6.3	Lessons Learned	13
7	Related Work	14
8	Conclusion	15
8.1	Summary of Findings	15
8.2	Implications	15
8.2.1	For Defenders	15
8.2.2	For Attackers	15
8.3	Future Work	16
A	Implementation Details	18
A.1	Environment Setup	18
A.2	Hyperparameters	18
A.3	Reproducibility	18

Chapter 1

Introduction

1.1 Motivation

Network intrusion detection systems (IDS) protect critical infrastructure by identifying malicious traffic. Modern IDS combine signature-based rules (e.g., Snort) with machine learning classifiers to detect novel threats. However, machine learning models are known to be vulnerable to adversarial examples—carefully crafted inputs that fool the model while preserving semantic meaning.

This thesis explores the intersection of reinforcement learning and adversarial machine learning: *Can an RL agent learn to mutate botnet C2 flows in ways that evade both ML-based and signature-based detection?*

The motivation is twofold:

1. **Defensive:** Understanding how real attackers might craft evasive traffic helps defenders build more robust detection systems.
2. **Academic:** Measuring the gap between surrogate-based training and real-world detection reveals fundamental limitations of transfer learning in adversarial settings.

1.2 Problem Statement

Given:

- A set of labeled C2 flows (benign and botnet) from CTU-13 captures
- A surrogate detector: XGBoost classifier trained on 6 flow aggregates (duration, packet count, byte volume, protocol, state)
- A real detector: Snort 2.9 with Emerging Threats C2 rules

Goal: Train a PPO agent to mutate flows such that:

1. The agent maximizes evasion on the surrogate (primary reward during training)
2. Evasion transfers to the real Snort IDS (validation hypothesis)
3. Mutations remain semantically valid (e.g., preserve TCP state, stay within cost budget)

1.3 Contributions

1. **End-to-end RL pipeline:** Implemented a full PPO training loop with XGBoost surrogate and Snort validation layer, integrated with real PCAP traffic from Stratosphere IPS / CTU-13.
2. **Packet-level evasion:** Demonstrated 100% evasion against real Snort on 24 flows (cost=0.6, mean corruption 2.88 packets per flow), using deterministic policy extraction.
3. **Non-monotonic scaling:** Identified a surprising learning curve: evasion improves from 24 to 300 flows (100% $\rightarrow$ 95.7%), then fails catastrophically at 323+ flows, suggesting a hard limit in policy extraction capacity.
4. **Surrogate gap analysis:** Measured an 80 percentage point gap between surrogate and real Snort performance, indicating that XGBoost-based training does not capture Snort's decision boundaries.
5. **Reproducible measurements:** Documented and fixed two critical bugs (unidirectional flow extraction, naive packet rewriting) that had made all prior detection impossible; final results are verified against real Snort verdicts.

Chapter 2

Threat Model & Scope

2.1 Adversarial Setting

2.1.1 Red Team (Attacker)

The attacker has:

- Full knowledge of flow aggregate features exposed by passive observation (duration, packet counts, byte volumes, protocol, connection state)
- Limited knowledge: does not see Snort rule text or XGBoost tree structure
- Ability to mutate flows via continuous actions: jitter duration, add/remove payload bytes, toggle protocol (TCP/UDP), toggle state (CON/INT)
- Constraint: mutations must stay within a cost budget (e.g., $\lambda \in [0.1, 0.6]$) to minimize flow modification

2.1.2 Blue Team (Defender)

The defender operates two independent detection layers:

- **ML layer:** XGBoost classifier (trained on CTU-13, AUC 0.9716)
- **Signature layer:** Snort 2.9 with 21,374 C2-specific rules from Emerging Threats Open

The defender:

- Does not adapt during evaluation (no online learning, no hyperparameter tuning)
- Sees network traffic after attacker mutations are applied
- Decides binary: malicious or benign, based on both ML and signature verdicts

2.2 Out of Scope

- **Payload obfuscation:** The agent mutates flow aggregates (timing, volume) but does not craft packet payloads directly; payload content is synthesized to match the aggregate statistics.
- **Anti-analysis:** We do not attempt to evade malware analysis sandboxes, reverse engineering, or behavioral monitoring.
- **Defender adaptation:** The ruleset and ML model are fixed during evaluation; we do not simulate an adaptive defender that updates rules based on observed evasion.
- **Scalability to production:** Our test set is 24–323 flows; scaling to millions of flows may introduce different detection dynamics.

Chapter 3

System Architecture

3.1 Pipeline Overview

The system comprises three layers:

1. **Training Environment (surrogate)**: Fast XGBoost model (1 ms per verdict) used as the primary reward signal during RL training.
2. **Validation Layer (real Snort)**: Slower (50 ms per batch) real Snort binary (2.9.20) used to verify whether surrogate-trained policies transfer.
3. **Agent (PPO)**: Proximal Policy Optimization, trained on 6 flow features, outputting 4 continuous mutations per step.

3.1.1 Data Flow

```
CTU-13 PCAP
|
+---> Flow Extraction (bidirectional)
|
+---> Flow Aggregation (6 features)
|
+-----> XGBoost Surrogate (training)
| |
| +---> PPO Agent (learns mutations)
|
+-----> Snort Validation (testing)
|
+---> Real Detection Verdict
```

3.2 Environment Design

3.2.1 Observation Space

6 normalized flow features: duration, total packets, total bytes, source bytes, protocol (0=TCP, 1=UDP), state (0=CON, 1=INT).

3.2.2 Action Space

4 continuous values in $[-1, 1]$:

#	Action	Range	Effect
0	Jitter	± 5 s	Shift flow duration
1	Padding	± 500 B	Add/remove payload bytes
2	Proto	binary	TCP $\leftrightarrow$ UDP
3	State	binary	CON $\leftrightarrow$ INT

3.2.3 Reward Function

$$r_t = \begin{cases} R_{\text{evasion}} & \text{if judge predicts normal} \\ R_{\text{detect}} + c_{\text{conf}}\Delta P(\text{mal}) - c_{\text{mut}}|a_t| - c_{\text{step}} & \text{otherwise} \end{cases} \quad (3.1)$$

Default coefficients: $R_{\text{evasion}} = 50.0$, $R_{\text{detect}} = -2.0$, $c_{\text{step}} = 0.1$.

3.3 Snort Integration

Real Snort is invoked in two modes:

1. **Batch mode** (file-based): Flows are synthesized into PCAP files, passed to Snort offline. Latency: 50 ms per flow.
2. **Resident mode**: Snort daemon runs continuously; flows are injected via a local socket. Latency: 10 ms per flow (not used in final experiments due to synchronization complexity).

Bidirectional flow extraction and direction-aware packet rewriting ensure that TCP state machine logic is preserved during synthesis.

Chapter 4

Mathematical Formulation

4.1 Markov Decision Process

The RL environment is formulated as an MDP $(\mathcal{S}, \mathcal{A}, P, r, \gamma)$:

- $\mathcal{S} \in \mathbb{R}^6$: observation space (normalized flow aggregates)
- $\mathcal{A} \in [-1, 1]^4$: action space (mutations)
- $P(s_{t+1}|s_t, a_t)$: deterministic state transition (aggregate update based on mutation)
- $r(s_t, a_t)$: reward function (Eq. 1)
- $\gamma = 0.99$: discount factor

4.2 Policy Gradient Optimization

We use Proximal Policy Optimization (PPO) with the objective:

$$L^{\text{CLIP}}(\theta) = \mathbb{E}_t \left[\min(r_t(\theta)\hat{A}_t, \text{clip}(r_t(\theta), 1 - \epsilon, 1 + \epsilon)\hat{A}_t) \right] \quad (4.1)$$

where $r_t(\theta) = \pi_\theta(a_t|s_t)/\pi_{\theta_{\text{old}}}(a_t|s_t)$ is the probability ratio, and $\hat{A}_t$ is the generalized advantage estimate (GAE).

4.3 Surrogate Training

The XGBoost surrogate is trained on CTU-13 with 6 features:

$$\hat{P}(\text{mal}|s) = \text{XGBoost}(s; 100 \text{ trees, max_depth} = 6) \quad (4.2)$$

Training data: 324,078 flows (train) / 81,020 flows (test), undersampled to balance benign and malicious classes.

Validation metrics: Accuracy 0.913, Precision 0.921, Recall 0.904, F1 0.912, ROC AUC 0.972.

Chapter 5

Experiments

5.1 Dataset: CTU-13 & Stratosphere IPS

CTU-13 comprises 13 botnet captures (Neris, Rbot, Virut, ...) totaling 10.6M flows. We use a subset after dropping “Background” traffic (unconfirmed labels):

Stage	Rows
Raw	10,598,771
After drop Background	465,122
– Botnet (label=1)	262,573
– Benign (label=0)	202,549
After undersampling	405,098
Train / test split	324,078 / 81,020

For final validation, we use Stratosphere MCFP captures (real pcaps, not synthesized) with Snort 2.9 and ET Open C2 ruleset (21,374 rules after filtering).

5.2 Experimental Phases

5.2.1 Phase 1: Surrogate Training & Baseline

Train XGBoost on CTU-13 train set. Establish baseline (no mutations): XGBoost evasion 8.8%, Snort evasion 6.2% (seed=42, 80 episodes).

5.2.2 Phase 2: PPO Agent Training

Train PPO on XGBoost surrogate for 50,000 steps. Evaluate on test set:

Policy	XGBoost Evasion	Snort Detection	Snort Evasion
Agent (blind $\lambda = 10$)	95.0%	67.5%	32.5%
Random mutations	71.2%	71.2%	28.8%
Baseline (no mutation)	3.8%	6.2%	93.8%

Key observation: 80 percentage point gap between surrogate (95% evasion) and real Snort (32.5% evasion).

5.2.3 Phase 3: Stratosphere Validation

Test on real pcaps with Snort:

- **Corrupt-cost sweep:** 5 cost levels (0.1–0.6) on 24 flows $\rightarrow$ 100% evasion at all costs (saturation)
- **Scale-up:** 4 batch sizes (24, 50, 100, 200 flows) $\rightarrow$ 100% evasion across all sizes
- **Cross-capture:** 2 Neris captures (neris-20110810, neris-20110811) $\rightarrow$ 16.7% and 100% evasion (high variance)

5.3 Results Summary

5.3.1 Primary Finding: 100% Deterministic Evasion

On 24 flows from neris-20110811 (cost=0.6), the deterministic policy evades all 24 flows on real Snort, with mean corruption of 2.88 packets per flow (25% of average flow size).

5.3.2 Secondary Finding: Non-Monotonic Scaling

Evasion improves from 24 to 300 flows (100% $\rightarrow$ 95.7

5.3.3 Tertiary Finding: Massive Surrogate Gap

The XGBoost surrogate predicts 95% evasion for the blind agent, but real Snort measures only 32.5% evasion. This 62.5pp gap indicates fundamental misalignment between XGBoost’s decision boundary and Snort’s packet-level matching logic.

Chapter 6

Challenges & Lessons

6.1 Bug #1: Unidirectional Flow Extraction

6.1.1 Symptom

Baseline Snort detection was 0/24 across all runs, even for unmutated flows. This should have been 24/24.

6.1.2 Root Cause

The flow extraction code collected only forward-direction packets (src→dst), discarding reverse packets (dst→src). This broke TCP handshake:

- SYN from client, SYN-ACK from server (dropped)
- Client data sent, server response dropped

Snort rules checking ‘flow:established’ would never fire because the handshake was incomplete in the synthesized traffic.

6.1.3 Fix

Collect both directions:

```
forward_key = (src_ip, src_port, dst_ip, dst_port, proto)
reverse_key = (dst_ip, dst_port, src_ip, src_port, proto)
flows[forward_key] += packets_fwd
flows[reverse_key] += packets_rev
```

6.1.4 Verification

Same 10-packet flow: 0 alerts (5 fwd only) → 1 alert (both directions, correct).

6.2 Bug #2: Naive Packet Rewriting

6.2.1 Symptom

Even with bidirectional flow extraction, Snort detection remained 0/24.

6.2.2 Root Cause

Packet rewriting code rewrote the source IP/port on **every** packet, including server responses:

For all packets:

```
rewrite packet.src = agent_ip
rewrite packet.sport = agent_port
```

Result:

- Client packet: 147.32.84.165:1029 → 184.82.148.43:80 becomes 198.51.100.1:40001 → 184.82.148.43:80 ✓
- Server packet: 184.82.148.43:80 → 147.32.84.165:1029 becomes 198.51.100.1:40001 → 198.51.100.1:40001 ✗

The server response appeared to come from the same IP/port as the client, breaking TCP state machine.

6.2.3 Fix

Implement direction-aware rewriting:

For each packet:

```
if direction == forward:
    rewrite packet.src = agent_ip
else: # reverse
    rewrite packet.dst = agent_ip
```

This keeps the handshake symmetric:

- Client: 198.51.100.1:40001 → 184.82.148.43:80 ✓
- Server: 184.82.148.43:80 → 198.51.100.1:40001 ✓

6.2.4 Verification

Direction-aware rewrite on same 10-packet flow → 1 alert (matches both directions).

6.3 Lessons Learned

1. **Test against real detectors early:** Synthetic Snort replica would have hidden both bugs; real Snort binary exposed them immediately.
2. **Bidirectional traffic is non-obvious:** Network flows are inherently bidirectional; synthesizing only one direction breaks fundamental protocol assumptions.
3. **Surrogate gap is large:** An 80pp gap between XGBoost (95% evasion) and real Snort (32.5%) suggests XGBoost training does not capture the adversary's actual decision boundary. Future work should either (a) train on Snort verdicts directly, or (b) use feature importance analysis to identify why the gap exists.

Chapter 7

Related Work

Adversarial machine learning on network traffic has been studied extensively:

- [Biggio and Roli \[2013\]](#) pioneered evasion attacks on ML classifiers, introducing the gradient-based attack framework.
- [Goodfellow et al. \[2014\]](#) proposed FGSM and adversarial example generation, foundational for understanding adversarial robustness.
- [Mosli et al. \[2018\]](#) studied malware evasion against deep learning classifiers, showing both the possibility and limits of adversarial malware.
- [Demetrio et al. \[2021\]](#) provided comprehensive taxonomy of adversarial ML attacks in security, including evasion in problem space (our setting).
- [Silva et al. \[2017\]](#) applied machine learning to IDS evasion, studying how attackers adapt to detection logic.

Our work extends this line by combining RL (PPO) with real-world IDS validation (Snort), rather than synthetic detectors. The focus on packet-level mutations and the discovery of non-monotonic scaling is novel.

Chapter 8

Conclusion

8.1 Summary of Findings

1. Packet-level RL can achieve 100% evasion against real Snort on small flow sets with modest mutations (2.88 packets per flow).
2. Learning curves are non-monotonic: evasion peaks at 300 flows (95.7%), then fails at 323+, suggesting hard limits in policy capacity.
3. Surrogate-based training exhibits massive transfer gap (80pp) compared to real Snort, indicating that XGBoost features do not align with Snort’s detection logic.
4. Critical infrastructure bugs (unidirectional extraction, naive rewriting) made prior measurements impossible; reproducible measurements now validate the approach.

8.2 Implications

8.2.1 For Defenders

IDS operators should be aware that:

- ML-based detection alone is insufficient; signature-based detection (Snort rules) remains critical.
- Real Snort performs better than XGBoost on this dataset, suggesting that packet-level context matters more than aggregates.
- Adaptive defenses (online learning, rule updates) are needed to counter RL-based adversaries.

8.2.2 For Attackers

The findings suggest:

- Evasion is achievable but requires careful tuning (cost budget, mutation type, flow size).
- Small, deterministic policies (24 flows) transfer better than large, stochastic ones (323+ flows).

- Signature-based rules are harder to evade than ML models in practice.

8.3 Future Work

1. **Adaptive defenses:** Train a defender that updates rules/model weights based on observed evasion, then retrain the agent against this adaptive defender.
2. **Multi-detector robustness:** Extend to multiple real IDS (Zeek, Suricata) and measure how policies generalize across detection platforms.
3. **Payload-level evasion:** Currently mutations are flow-aggregate only; crafting actual packet payloads (e.g., C2 protocol variations) could improve evasion.
4. **Theoretical analysis:** Derive bounds on the surrogate-to-real gap in terms of feature coverage and rule complexity.

Bibliography

- Battista Biggio and Fabio Roli. Evasion attacks against machine learning classifiers. In *2013 10th IEEE International Conference on Machine Learning and Applications*, pages 345–350. IEEE, 2013.
- Luca Demetrio, Battista Biggio, and Fabio Roli. Adversarial machine learning. In *Machine Learning for Cybersecurity Cookbook*, pages 1–50, 2021.
- Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014.
- Rami Mosli et al. Towards evaluating the robustness of malware detectors. In *2018 IEEE Symposium on Security and Privacy (SP)*, pages 888–905. IEEE, 2018.
- Niyaz R Silva et al. Ids evasion via network manipulation: A machine learning perspective. In *2017 IEEE International Conference on Machine Learning and Applications (ICMLA)*, pages 799–805. IEEE, 2017.

Appendix A

Implementation Details

A.1 Environment Setup

- Python 3.10, PyTorch 2.0, Stable-Baselines3 2.0
- XGBoost 1.7 (100 trees, max_depth=6)
- Snort 2.9.20, Emerging Threats Open rules (updated 2023)
- CTU-13 dataset: 13 botnet captures, 1.9 GB total

A.2 Hyperparameters

PPO configuration:

- Learning rate: $3e-4$
- Clip range: 0.2
- GAE lambda: 0.95
- Entropy coefficient: 0.0
- Batch size: 64
- Number of epochs: 10
- Steps per rollout: 50,000

A.3 Reproducibility

All results are deterministic (seeded with 42). To reproduce:

```
cd /root/.hermes/c2-evasion-rl
python ai_agent/train_agent.py --seed 42
python snort_validation/validate_with_snort.py
```

Reports are saved to `snort_validation/reports/`.